

HARNESS ENGINEERING

하네스 엔지니어링 실습 커리큘럼

Karpathy × OpenAI × Mitchell Hashimoto

세 소스를 통합한 5모듈 이론 + 실습 과정

Karpathy CLAUDE.md

OpenAI Harness Eng.

Mitchell Hashimoto

01

02

03

04

05

세 가지 핵심 소스

KARPATY CLAUDE.md

선언 층

에이전트에게
무엇을 기대하는가

4원칙 : Think Before Coding /
Simplicity First / Surgical Changes /
Goal-Driven Execution

OPENAI HARNESS ENG.

인프라 층

어떤 환경을
만드는가

Context Engineering · Architectural
Constraints · Entropy Management ·
Lifecycle Scaffolding

MITCHELL HASHIMOTO

피드백 루프

어떻게
진화시키는가

"실패할 때마다 시스템을
엔지니어링하라" — 실패를
규칙으로, 규칙을 코드로

5모듈 커리큘럼 로드맵

이론 설명 + 프로젝트 직접 적용 실습

01

왜 프롬프트로는
부족한가

등장 배경 +
베이스라인 측정

이론

실습

02

CLAUDE.md
에이전트 헌법

Karpathy 4원칙 적용

이론

실습

03

Hooks
시스템 강제

guard.sh + 자기검증
루프

이론

실습

04

멀티 에이전트
컨텍스트 설계

Planner / Coder / Critic

이론

실습

05

하네스 진화
실패를 자산으로

엔트로피 관리 + Drift
방지

이론

실습

왜 프롬프트로는 부족한가

하네스 엔지니어링의 등장 배경과 핵심 개념

Hashimoto

OpenAI

Karpathy

이론

프롬프트 → 컨텍스트 → 하네스 진화

2023 단발 지시 / 2025 맥락 설계 / 2026 환경 설계

말(AI)과 마구(Harness) 비유

"절대 하지 마" 프롬프트가 왜 실패하는가

Hashimoto 원칙

"실패할 때마다 시스템을 엔지니어링하라"

OpenAI 5개월 실험

100만 줄 코드, 인간 코드 0줄 — 환경이 병목이었다

실습

실패 패턴 감사 (Failure Audit)

[실습 →](#)

현재 Claude Code 작업에서 반복되는 실수 3가지 찾기. 각각 "프롬프트로 막을 수 있는가 vs 시스템으로 막아야 하는가" 분류.

베이스라인 측정

[실습 →](#)

하네스 없이 에이전트에게 태스크 3개 맡겨보기. 성공률·불필요한 변경·방향 이탈을 기록해 기준점 설정.

CLAUDE.md vs AGENTS.md 비교

[실습 →](#)

두 파일의 차이와 언제 어느 것을 쓸지 이해하고, 프로젝트에 맞는 기본 골격 작성.

Karpathy 4원칙을 Spring Boot DDD 프로젝트에 적용

이론

Think Before Coding

가정 명시 / 혼란 숨기지 않기 / 더 단순한 방법 먼저 말하기

Simplicity First

요청된 것만. 200줄이 50줄이면 다시 써라.

Surgical Changes

내 변경이 만든 orphan만 정리. 기존 dead code는 언급만.

Goal-Driven Execution

[단계] → verify: [확인법] 형식. 성공 기준을 먼저 정의하라.

실습

CLAUDE.md 초안 작성

[실습 →](#)

Karpathy 4원칙 + 프로젝트 스택 규칙 통합. 절대 금지 트리거(migration 삭제, Entity 직접 노출 등) 포함. 500줄 이하 유지.

Before / After 비교

[실습 →](#)

CLAUDE.md 적용 전후로 동일 태스크 실행. diff 크기와 불필요한 변경 횟수 비교해서 효과 측정.

실패 패턴 → 규칙 변환

[실습 →](#)

모듈1에서 찾은 실패 패턴을 CLAUDE.md 금지 규칙으로 전환. 한 줄씩 추가하며 진화.

프롬프트를 '부탁'에서 '구조'로 — 에이전트 라이프사이클 제어

이론

에이전트 라이프사이클 이벤트

세션 시작 / 도구 실행 전·후 / 응답 완료 시점에 셸 스크립트 주입

guard.sh — 위험 명령 차단

Bash 실행 전 실행 → `rm -rf`, `DROP TABLE`, `migration` 수정 등 필터링

lint-fix.sh — 자동 포맷 복구

파일 수정 후 에이전트가 스타일 규칙 위반 시 자동 `checkstyle` 실행

Back-pressure 메커니즘

타입체크, 테스트, 커버리지를 에이전트 자기검증 도구로 연결

실습

Hooks 설정 구현

[실습 →](#)

`guard.sh` + `lint-fix.sh` 작성. Claude Code hooks 설정 파일에 연결. 실제로 금지 명령이 차단되는지 테스트.

자기검증 루프 구축

[실습 →](#)

`./gradlew test` 결과를 에이전트 검증에 포함. 테스트 실패 시 에이전트가 스스로 수정하는 루프 완성.

Hooks 진화 실습

[실습 →](#)

실습 중 발생한 새 실패를 `guard.sh` 규칙으로 추가. "다음엔 잘 해줘" 대신 구조적으로 방지.

 이론**컨텍스트 방화벽으로서의 서브에이전트**

프론트/백 분리가 아니라 컨텍스트 윈도우 오염 방지가 목적

3-tier 컨텍스트 인프라

CLAUDE.md(전역) / 스킬 파일(태스크) / claude-progress.txt(세션)

Critic 에이전트 패턴

Claude + Codex 3라운드 토론 → CONDITIONAL REJECT 권한 부여

AGENTS.md — 모델 불가지론적 표준

CLAUDE.md를 넘어 어떤 모델이든 읽을 수 있는 공용 표준

 실습**3-tier 워크플로우 설계**[실습 →](#)

task-list.md 기반 Planner-Coder-Critic 워크플로우. 지방계약 관리 시스템 실제 태스크로 드라이런.

세션 인계 프로토콜[실습 →](#)

claude-progress.txt + git log 기반 상태 복원 프로토콜. 새 컨텍스트 윈도우에서 5초 안에 현재 상태 파악.

Critic 에이전트 실습[실습 →](#)

구현 계획을 Critic에게 보여주고 DDD 원칙 위반 여부 검증. CONDITIONAL REJECT 처리 흐름 완성.

하네스 진화 — 실패를 자산으로

OpenAI

Hashimoto

엔트로피 관리와 지속적 개선 루프 확립

이론

하네스는 저장소에 커밋되는 자산

팀원·모델이 바뀌어도 CLAUDE.md + hooks + skills는 남는다

엔트로피 관리 — Drift 방지

장기 프로젝트에서 에이전트 출력이 점점 발산하는 문제 대응

과적합 함정 (Rippable Harness)

Opus 4.6: 자기 하네스 33위 → 다른 하네스 5위권. 모델 개선 시 일부 하네스는 버려야 한다

주간 하네스 리뷰 루틴

실패 패턴 → 규칙 추가 사이클. 매주 CLAUDE.md 한 줄 이상 추가 목표

실습

하네스 저장소 구조화

[실습 →](#)

.claude/ 폴더에 CLAUDE.md + hooks + skills 정리. README에 하네스 사용법 문서화. 팀 온보딩 체크리스트 작성.

주간 하네스 리뷰 실습

[실습 →](#)

지난 1주일 Claude Code 실패 패턴 3개 수집. 각각 CLAUDE.md 규칙 또는 guard.sh 항목으로 전환.

Rippable 하네스 점검

[실습 →](#)

현재 CLAUDE.md 규칙 중 모델 개선으로 이미 불필요해진 항목 제거. 하네스 최적화.

핵심 원칙 요약

01

부탁 대신 구조

프롬프트는 부탁이다.
시스템이 강제한다.
guard.sh가 막으면
에이전트는 못 한다.

02

실패가 규칙이 된다

같은 실수 두 번 → 즉시
CLAUDE.md 추가. "다음엔
잘 해줘"는 없다.

03

검증 가능한 목표

[단계] → verify: [확인법].
성공 기준이 없으면
에이전트도 멈춘다.

04

하네스는 자산이다

CLAUDE.md + hooks + skills =
팀이 쌓는 경쟁 우위.
모델이 바뀌어도 남는다.

모델을 실행하기 전에 하네스를 점검하라

— *Mitchell Hashimoto*

CLAUDE.md가 무엇을 담고 있는가 · 어떤 Hooks를 설정했는가 · 피드백 루프가 있는가